Володимирський педагогічний фаховий коледж імені Агатангела Кримського
циклова комісія викладачів професійної цифрової та математичної освіти

Інструкція до практичної роботи №51

ТЕМА: Обробка подій та зв'язування методів (command, bind()). Індикатори та повзунки (CTkSlider, CTkProgressBar).

Освітньо-професійна програма: Професійна освіта (Цифрові технології)
Тривалість заняття: 80 хв

Мета заняття:

- поглибити знання про обробку подій у графічному інтерфейсі;
- вивчити відмінність між параметром command і методом bind();
- ознайомитися з віджетами CTkSlider (повзунок) та CTkProgressBar (індикатор прогресу);
- навчитися пов'язувати дії користувача зі зміною стану інших віджетів у реальному часі.

Теоретичні відомості

1. Подія в графічному інтерфейсі

Подія (event) — це будь-яка дія користувача, яку програма може відстежити: натискання кнопки миші, рух повзунка, введення символу в поле, наведення курсора на елемент, натискання клавіші на клавіатурі. Уся робота графічної програми, по суті, зводиться до очікування подій (у циклі mainloop()) і виконання відповідного коду-обробника щоразу, коли потрібна подія відбувається.

2. Параметр command — найпростіший спосіб обробки подій

Параметр command, з яким ми вже працювали для кнопок, — це найпростіший спосіб пов'язати віджет із функцією-обробником. Він підтримується більшістю інтерактивних віджетів (CTkButton, CTkCheckBox, CTkSwitch, CTkOptionMenu тощо) і викликає вказану функцію щоразу, коли відбувається головна, «очікувана» дія для цього віджета (клік для кнопки, зміна стану для перемикача).

Обмеження command полягає в тому, що він реагує лише на один, наперед визначений тип події, характерний для конкретного віджета, — наприклад, для CTkButton це лише клік, і command не дозволяє, скажімо, окремо обробити наведення курсора чи натискання конкретної клавіші.

3. Метод bind() — гнучке зв'язування подій

Метод bind() дозволяє пов'язати з віджетом (або навіть із цілим вікном) обробник для практично будь-якого типу події, вказаного спеціальним рядком-ідентифікатором:

Ідентифікатор події	Коли спрацьовує
'<Button-1>'	клацання лівою кнопкою миші
'<Double-Button-1>'	подвійне клацання лівою кнопкою миші
'<Enter>'	курсор наводиться на віджет
'<Leave>'	курсор залишає межі віджета
'<Key>'	натискання будь-якої клавіші клавіатури
'<Return>'	натискання клавіші Enter

```
def on_enter(event):
    label.configure(text='Курсор тут!')

def on_leave(event):
    label.configure(text='Наведіть курсор')

label = ctk.CTkLabel(app, text='Наведіть курсор', width=200,
height=60)
label.pack(pady=20)
label.bind('<Enter>', on_enter)
label.bind('<Leave>', on_leave)
```

Важлива відмінність від command: функція-обробник, прив'язана через bind(), автоматично отримує один аргумент — об'єкт event, що містить додаткову інформацію про подію (наприклад, координати кліку миші). Навіть якщо ця інформація не потрібна, параметр event все одно обов'язково має бути в оголошенні функції-обробника.

4. CTkSlider — повзунок вибору числового значення

CTkSlider дозволяє користувачу плавно обирати числове значення в заданому діапазоні, пересуваючи повзунок мишею. Діапазон задається параметрами from_ і to, а параметр command викликається щоразу під час перетягування повзунка, отримуючи поточне значення як аргумент:

```
def on_slide(value):
    volume_label.configure(text=f'Гучність: {int(value)}%')

volume_label = ctk.CTkLabel(app, text='Гучність: 50%')
volume_label.pack(pady=(20, 5))

slider = ctk.CTkSlider(app, from_=0, to=100, command=on_slide)
slider.set(50)
slider.pack(pady=10)
```

Метод set() встановлює початкове значення повзунка програмно (наприклад, під час запуску програми), а get() дозволяє будь-коли дізнатися поточне значення, навіть поза функцією-обробником command.

5. CTkProgressBar — індикатор прогресу

CTkProgressBar показує наочний індикатор виконання якогось процесу (завантаження файлу, обробки даних) у вигляді смуги, що заповнюється. На відміну від повзунка, індикатор прогресу зазвичай не призначений для взаємодії користувача — його значення встановлюють програмно методом set() із числом від 0 (порожній) до 1 (повністю заповнений):

```
progress = ctk.CTkProgressBar(app, width=250)
progress.set(0)
progress.pack(pady=20)

def simulate_progress():
    value = progress.get()
    if value < 1:
        progress.set(value + 0.1)
        app.after(200, simulate_progress)

ctk.CTkButton(app, text='Почати', command=simulate_progress).pack()
```

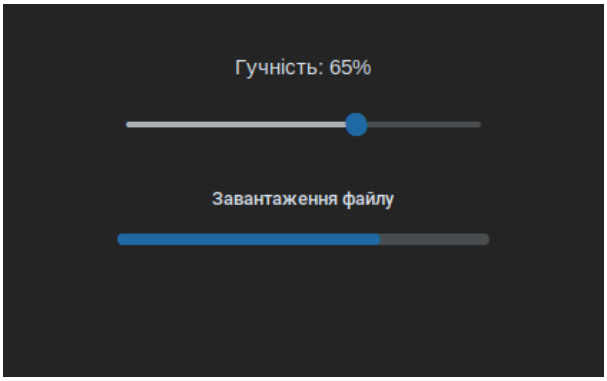


Рис. 1. CTkSlider (повзунок гучності) та CTkProgressBar (індикатор завантаження)

У прикладі вище використано метод app.after(200, simulate_progress) — він планує повторний виклик тієї самої функції через 200 мілісекунд, і так доти, доки значення не досягне 1. Такий прийом («рекурсивне» планування через after()) є стандартним способом створити плавну, покрокову анімацію в tkinter/CustomTkinter без блокування головного циклу подій.

Хід роботи

Варіант і вимоги до здачі

а) N — ваш порядковий номер у списку групи (журнал). Затримка в завданні 4 і мета в завданні 6* залежать від N (mod — остача від ділення):

Завдання	**Дані варіанта**	**Приклад N = 7**
4	затримка D = 20 + 5·(N mod 4) мс	D = 35
6*	мета G = 50 + 10·(N mod 5) кліків	G = 70

б) Кожне завдання — окремий файл Прізвище_ПР51_зN.py (наприклад, Ivanenko_PR51_z3.py), код текстом, а не фото чи скріншотом. Файли з прикладів із теорії та додаткові файли, яких немає в умові, здавати не потрібно.

в) Запускайте програми в IDLE або іншому середовищі на своєму комп'ютері; до звіту (документ Word) додайте скріншот виконання кожної програми.

г) Для кожного вікна додайте до звіту скріншот із видимим заголовком вікна. Імпорт — import customtkinter as ctk; програма завершується app.mainloop(). Тексти елементів (кнопок, міток, повідомлень у консоль) — точно як в умові.

д) У коді залиште короткі коментарі власними словами перед основними блоками.

е) Будьте готові пояснити будь-який рядок свого коду й внести в нього невелику зміну (див. розділ «Захист роботи»).

1. Створіть файл Прізвище_ПР51_з1.py. Прив'яжіть до CTkLabel обробники '<Enter>' і '<Leave>' (bind()), що змінюють текст мітки на «Курсор над міткою» і «Курсор поза міткою».

   Приклад: наведення курсора змінює текст, відведення — повертає.

2. Додайте обробник '<Button-1>' до мітки, що виводить у консоль «Клік: x=…, y=…» (event.x, event.y).

   Приклад: клік у лівий верхній кут → «Клік: x=3, y=4» (значення залежать від місця кліку).

3. Додайте CTkSlider від 0 до 200 і мітку «Значення: …», що в реальному часі показує ціле значення.

   Приклад: повзунок посередині → «Значення: 100».

4. Додайте CTkProgressBar і кнопку «Почати»: через after() індикатор заповнюється від 0 до 1 кроком 0.01 із затримкою D мс вашого варіанта.

   Приклад (N = 7): D = 35 мс; повне заповнення приблизно за 3.5 с.

5. Повзунок від 0 до 100 керує індикатором у реальному часі: set(значення / 100).

   Приклад: повзунок 50 → індикатор заповнений наполовину.

6. Завдання підвищеної складності*. «Клікер»: мітка «Кліків: k» збільшується на 1 при кожному кліку по великій кнопці; CTkProgressBar

показує k / G, де G — мета вашого варіанта; після досягнення мети виведіть «Мету досягнуто!».

Приклад (N = 7): G = 70; при 35 кліках індикатор 0.5; після 70 кліків — «Мету досягнуто!».

Критерії оцінювання

Складова	Оцінювання	Макс. балів
Завдання 1–5	кожне по 1,8 бала	9
Завдання 6*	2 бали	2
Контрольні питання	відповіді у звіті	1
Разом		**12**

Кожне завдання оцінюється так: програму виконано повністю, вона працює на вимогах умови й прикладах формату виводу — 100% балів; програма працює, але є суттєве зауваження (наприклад, після досягнення мети лічильник росте далі, обробник не приймає event, значення повзунка не цілі, анімація блокує вікно (цикл замість after())) — 50%; програма не працює або не здана — 0. Завдання 1–5 оцінюються по 1,8 бала, завдання 6* — 2 бали. Контрольні питання оцінюються в 1 бал, якщо на всі питання дано правильні відповіді.

Захист роботи

Перед виставленням оцінки викладач може попросити вас пояснити будь-який рядок свого коду, показати, як змінюється значення змінних на прикладі, або внести в програму невелику зміну (наприклад, змінити подію чи затримку/мету). Якщо пояснити код не вдається, оцінку за відповідне завдання може бути знижено.

Контрольні питання

Відповідайте на питання письмово у звіті (документ Word разом із скріншотами): 1–2 речення на кожне питання, власними словами.

1. Чим command відрізняється від bind() за гнучкістю обробки подій?
2. Який додатковий аргумент автоматично отримує функція, прив'язана через bind()?
3. Які значення приймає метод set() у CTkProgressBar?
4. Як дізнатися поточне значення повзунка CTkSlider поза функцією command?
5. Для чого в прикладі анімації прогрес-бару використано метод app.after()?